

10 — Knowledge Base

Objetivo. La *Knowledge Base* es la **memoria colectiva** de Puglit: el conjunto de estructuras en Postgres (más sus espejos en Obsidian y git) donde el enjambre acumula lo que aprendió build tras build. No es documentación humana — es *estado entrenable* que se inyecta de vuelta en los prompts de la próxima generación. Acá viven los patrones que funcionaron, los que fallaron, los módulos reutilizables, las skills evolucionadas y las métricas que validan la tesis por evidencia y no por mecanismo.

La memoria se reparte en cuatro tablas-cerebro de `web/sql/genetic.sql`, cada una con una función distinta dentro del ciclo de aprendizaje:

Tabla	Qué guarda	Cómo se recupera	Archivo de lógica
<code>verified_exemplars</code>	Código (route/page) de builds que pasaron el gate	Por similitud semántica (floor 0.55)	<code>web/lib/swarm-fitness.ts</code>
<code>puglit_agent_diary</code>	Lecciones/wins/críticas por agente, con embeddings	Por relevancia + decay (floor 0.35)	<code>web/lib/progression.ts</code>
<code>puglit_modules</code>	Módulos cosechados/mejorados por el enjambre	Por keyword, sólo <code>stable / core</code>	<code>web/lib/module-registry.ts</code>
<code>puglit_metrics</code>	KPIs de evidencia (build/smoke/judge/ablation)	Agregación por nombre + ventana	<code>web/lib/swarm-metrics.ts</code>
<code>puglit_skills</code> / <code>puglit_skill_rejects</code>	Skill docs evolucionadas + buffer de rechazos	Overlay activo por área (<code>skillFor</code>)	<code>web/lib/skill-evolution.ts</code>

Nota de honestidad metodológica. Lo que hoy se mide y persiste **automáticamente** es lo que tiene una tabla detrás: `puglit_metrics` (4 KPIs), `verified_exemplars`, `puglit_agent_diary`, `puglit_modules` y `puglit_skills`. Las secciones de *rankings* (módulos más usados, patrones más exitosos, timelines, insights semanales/mensuales) describen lo que la memoria **contiene** y lo que es **agregable** sobre esas tablas; varias de esas agregaciones todavía **no tienen un job dedicado** que las materialice. Donde corresponde lo aclaramos con la marca **[agregación futura]**. Nada acá inventa features que no estén en el código.

10.1 Top Successful Patterns

Los patrones exitosos se materializan de dos formas complementarias. La primera es **estructural y determinística** — los `deterministicX(config, bp)` del *module-registry* que inyectan código *correcto-*

por-construcción cuando el blueprint dispara su keyword (ver `web/lib/module-registry.ts`). La segunda es **empírica** — los `verified_exemplars`: pedazos de código de builds reales que **pasaron el runtime gate** y se guardan para que el modelo *imite código que funciona en vez de inventarlo*.

Patrones determinísticos (correctos por construcción)

Estos no se "aprenden": se inyectan ya correctos. Son el piso de calidad del que parte cualquier build.

Patrón	Módulo	Por qué es un "successful pattern"
Anti-double-booking	<code>rentals</code> , <code>booking</code>	Constraint Postgres <code>EXCLUDE USING gist</code> con rango half-open — imposible reservar dos veces a nivel DB, no a nivel app
Pricing en enteros (cents)	<code>rentals</code>	<code>priceQuote</code> determinístico, <code>search == checkout</code> (sin drift de redondeo)
Ledger append-only	<code>wallet</code> , <code>inventory</code>	<code>balance = sum(...)</code> no puede driftear; débito atómico sin negativos
Fire-and-forget seguro	<code>queue</code>	<code>FOR UPDATE SKIP LOCKED</code> + retries/backoff; prohíbe loops paralelos in-process
Secrets cifrados	<code>crypto</code> (co-inyectado)	AES-256-GCM at rest; se auto-inyecta con <code>payments</code> / <code>social-auth</code>
Audit hash-chained	<code>auditlog</code>	Filas encadenadas por hash → tamper-evident

Exemplars verificados (aprendidos de builds reales)

`storeExemplar(kind, task, code)` guarda sólo código de **80–8000 chars** que vino de un build que pasó el gate; lo embebe con el `task` como clave. `exemplarFor(kind, taskText)` lo recupera con un **floor de similitud de 0.55** — sólo un ejemplo *genuinamente* parecido entra al prompt, y entra con la instrucción explícita de *imitar la estructura, no copiar el dominio*.

```
storeExemplar("route", task, code)  → INSERT verified_exemplars (kind, task, code, embedding)
                                     (gate de longitud 80..8000, embedding de nomic-embed)

exemplarFor("route", taskText)      → embed(taskText) → cosine vs últimos 60 del kind
                                     → mejor match si sim > 0.55, recortado a 3000 chars
```

A esto lo acompaña la presión de selección objetiva: `objectiveScore(bp)` (0–100, **sin LLM**) premia schemas reales (cada tabla con ≥ 2 columnas: +14), cobertura de operaciones (routes $\geq$ tablas: +14) y penaliza *phantom tables* (−4 por cada tabla referenciada y nunca declarada, hasta −20). Ese score entra al torneo con peso 40% (el veredicto LLM pesa 60%, ver `web/lib/tournament.ts`), así la "fitness" no depende sólo del juez ruidoso.

10.2 Top Failed Patterns

Los patrones que fallan están **codificados como detectores** — el conocimiento de "esto sale mal" no vive en prosa sino en escáneres que los cazan y reparadores que los arreglan. Salieron del *litmus test* (Stayforge, el clon de Airbnb) donde el runtime gate expuso **7 bugs reales invisibles a un compile verde**.

Failed pattern	Síntoma	Dónde se caza	Mitigación
Phantom tables	SQL referencia una tabla nunca <code>CREATE</code> -ada (schema alucinado)	<code>swarm-checks.ts</code> (<code>phantom-table</code> , severity high) + penalización en <code>objectiveScore</code>	<code>swarm-repair.ts</code> : decide si es typo/alias de una tabla intencionada (mapea, no inventa) o infiere columnas del uso
Hardcoding	Literales mágicos, nombres de tabla/columna, IDs, precios, límites hardcodeados	Playbook <code>DEV</code> ("NEVER HARDCODE") + crítica del completeness-critic / REVIEW	Módulos parametrizados a la tabla detectada (ej. <code>rentals</code> zero-hardcoding); flag en review
<code>pool()</code> bug	Llamar al pool como función en vez de usar <code>pool.query</code> / mal import de <code>db</code>	Spine canónico: <code>export const pool = new Pool() (spine/)</code>	El spine determinístico provee el <code>db</code> correcto; los agentes lo importan, no lo reescriben
JSON over-escape	El modelo sobre-escapa JSON (doble escape) y rompe el parseo	Defensa en parseo (try/catch + fallback) en <code>relevantLessons</code> / <code>exemplarFor</code> al leer <code>embedding</code>	Lectura tolerante: <code>Array.isArray(v) ? v : JSON.parse(...)</code> con <code>catch</code> → <code>[]</code>
Hardcoded secret	<code>sk-...</code> , <code>AKIA...</code> , <code>ghp_...</code> en el código	<code>swarm-checks.ts</code> regex de secretos (severity high)	Escalado a frontier si el local no lo arregla; co-inyección de <code>crypto</code>
Schema/route name mismatch	El route usa un nombre de tabla distinto al del schema	Playbook <code>REVIEW</code> (coherencia & naming: una concepto = un nombre)	Flag de "ONE name per concept"

Anti-poisoning como failed-pattern de la propia memoria. Un riesgo de meta-nivel es que una lección *de un build malo* contamine los futuros. `relevantLessons` lo previene: sólo recupera lecciones con `quality ≥ 45` y `outcome ≠ 'failure'` , con un **floor de relevancia 0.35** (una lección de fintech no se fuerza en un build de health) y **decay de recencia** (half-life ~31 días). Si nada supera el floor, devuelve vacío: "*no advice beats wrong advice*".

10.3 Common Build Errors

Las clases de error que el sistema reconoce y a las que reacciona. Las severidades vienen de `swarm-checks.ts` ; la disposición final (SHIP / SHIP-WITH-CAVEATS / BLOCK) de `adversarial-review.ts` .

Clase de error	Severidad	Detector
<code>phantom-table</code>	high	<code>securityScan</code> / consistency scan
<code>sql-injection</code>	high	SQL armado por concatenación de strings
<code>hardcoded-secret</code>	high	regex de keys reales committeadas
<code>dangerous-exec</code>	high	<code>eval</code> / <code>exec</code>
<code>hardcoded-credential</code>	med	credencial = string literal (en vez de <code>process.env</code>)
Runtime 5xx	bloqueante	<code>web/scripts/build-local.mjs</code> — bootea la app y smoke-testea las páginas (gate: 0 5xx)
Cobertura de tests por debajo de la barra	bloqueante	QA gate: vitest + coverage + Queen evidence review (mide cobertura real, rebota)

El insight central del litmus test: **un compile verde no es evidencia**. El *runtime gate* de `build-local.mjs` arranca la app y prueba las páginas; ahí aparecieron los 7 bugs de Stayforge que TypeScript no veía.

10.4 Common Repair Actions

Las reparaciones son **acotadas y determinísticas** primero, con escalada a un modelo más fuerte sólo para lo que el local no puede.

Acción de reparación	Trigger	Lógica
Phantom-table reconcile	<code>phantom-table</code> detectada	<code>swarm-repair.ts</code> : el reparador decide tipo→alias (<code>CREATE VIEW alias AS SELECT * FROM real_table</code>) vs tabla nueva (infiere columnas/tipos/keys del uso). La intención del arquitecto es ground-truth
Frontier escalation	Bug que el local no arregla (ej. seguridad)	<code>frontierEscalate()</code> — presupuesto acotado por build (<code>PUGLIT_FRONTIER_BUDGET</code>), modelo en <code>PUGLIT_FRONTIER_MODEL</code> , temp 0.1; devuelve <code>null</code> si no hay budget/config
Adversarial bounded repair	3 lentes (Auditor/Adversary/Pragmatist) confirman (≥ 2) un problema	<code>adversarial-review.ts</code> → reparación acotada o BLOCK
Best-of-N (rejection sampling)	Generación ruidosa	<code>generateBest(gen, score, N)</code> corre N veces y se queda con el de mayor objective score (<code>PUGLIT_BEST_OF</code>)
Lesson injection	Próximo prompt del agente	<code>relevantLessons</code> / <code>recentLessons</code> inyectan las lecciones diary como few-shot

10.5 Most Used Modules

El catálogo builtin tiene ~85 módulos (`BUILTIN_MODULES` en `web/lib/module-registry.ts`), agrupados por categoría. Un módulo se "usa" cuando su `deterministicX` se inyecta en *finalize* porque el blueprint disparó su keyword, o cuando aparece como dependencia dura vía `MODULE_REQUIRES` .

Distribución del catálogo por categoría (conteo real sobre `BUILTIN_MODULES`):

Categoría	Aprox.	Ejemplos
<code>util</code>	mayoría	queue, storage, cache, stats, charts, wallet, validation, booking, reviews
<code>integration</code>	~25	payments, billing, nango, crm, ecommerce, sso, bi, shipping
<code>channel</code>	~7	telegram, email, whatsapp, slack, discord, teams
<code>agent</code>	~10	llm, rag, agent (JARVIS), voice, graphify, memorygraph, docparse, ocr

Los módulos con **más entrada por dependencia** (los que se co-inyectan más seguido por `MODULE_REQUIRES` → `dependencyClosure`) son estructuralmente los más "usados":

Módulo base	Lo arrastran
<code>crypto</code>	social-auth, billing, payments, agent
<code>llm</code>	moderation, rag, agent, graphify
<code>storage</code>	imagegen, media
<code>realtime</code>	inappnotify
<code>stats</code>	charts
<code>validation</code>	forms

[Agregación futura]. El conteo de *cuántas veces* cada módulo se inyectó en builds reales requiere agregar sobre las trazas de build (`run-trace.ts` / `puglit-metrics.meta`). Hoy ese ranking no se materializa en una tabla; lo que es real y verificable es el **catálogo** y el **grafo de dependencias** que define qué se co-inyecta.

10.6 Most Successful Modules

"Exitoso" = el módulo que, cuando se inyecta, **no genera bugs de runtime** porque su corrección está garantizada por construcción (constraint DB, atomicidad, idempotencia) en vez de depender del LLM.

Módulo	Garantía de corrección
<code>rentals</code>	Constraint <code>EXCLUDE USING gist</code> , pricing integer-cents, refund-by-policy-snapshot, double-blind reviews — parametrizado a la tabla detectada (zero hardcoding). Ships <code>scripts/verify-rentals.mjs</code> + <code>/api/health</code>
<code>wallet</code>	Ledger append-only, balance = sum (no puede driftear), débito atómico sin negativos
<code>queue</code>	<code>FOR UPDATE SKIP LOCKED</code> + retries/backoff — el fire-and-forget seguro
<code>inventory</code>	Reserva atómica, sin overselling, vía ledger append-only
<code>auditlog</code>	Filas hash-chained (tamper-evident)
<code>booking</code>	<code>freeSlots()</code> + <code>book()</code> con prevención de double-booking

El denominador común: estos módulos llevan **su propia prueba** (ej. `verify-rentals.mjs`) y mueven la corrección al nivel del schema/constraint, donde el modelo no la puede romper.

[Agregación futura]. Un ranking de éxito *medido* (smoke-pass-rate condicionado a tener el módulo X) saldría de cruzar `puglit_metrics.smoke_pass` con la lista de módulos inyectados por build. La infraestructura de métricas existe (`recordMetric` / `metricRate`); el corte por módulo todavía no se computa.

10.7 Most Problematic Modules

Los más problemáticos son, por diseño, los que dependen de **un gateway externo** o de **un LLM** — ahí entra la varianza que los módulos determinísticos eliminan.

Fuente de problema	Módulos afectados	Por qué
Dependen de gateway Docker	scraper, whatsapp, pdf, media, docgen, ocr, crm, ecommerce, sso, bi, n8n, ...	El build puede estar perfecto pero el sidecar no estar levantado/configurado
Dependen de LLM (varianza)	moderation, rag, agent, graphify	Su salida no es determinística; entran en <code>MODULE_REQUIRES.llm</code>
Riesgo legal / gris	<code>scraper</code> (cookies de sesión, LinkedIn/paywalls)	Ban risk explícito en su propia descripción
Harvest sin gobernanza	módulos auto-cosechados (<code>harvestModules</code>)	Entran como <code>experimental</code> ; <code>findCustomModulesFor filtra</code> y sólo deja pasar <code>stable</code> / <code>core</code> para que no contaminen builds reales (crítica: catalog rot)

El ciclo de vida de harvest es la mitigación central: **experimental** → **candidate** → **stable** → **core**. Un módulo cosechado del trabajo del propio enjambre no toca un build de producción hasta que un humano o un proceso lo **promueve** (`promoteModule`). Eso convierte "módulo problemático" en un estado *transitorio* y *aislado*, no en deuda.

10.8 Evolution Timeline

Las mejoras de esta línea de trabajo, cada una atada a su archivo y a la crítica que la motivó. Conceptualmente es la "historia evolutiva" de la base de conocimiento.

Hito	Qué introdujo	Archivo
No-hardcode discipline	Playbook DEV con "NEVER HARDCODE"; módulos verticales parametrizados a la tabla detectada (rentals)	web/lib/playbooks.ts , web/lib/module-registry.ts
QA / coverage gate	Runtime gate que bootea la app (0 5xx) + vitest + coverage + Queen evidence review que mide cobertura real y rebota debajo de la barra	web/scripts/build-local.mjs
Adverse (adversarial review)	Patrón addyosmani/adverse: 3 lentes ortogonales (Auditor/Adversary/Pragmatist) → cross-examination (≥2 = confirmado) → SHIP / SHIP-WITH-CAVEATS / BLOCK	web/lib/adversarial-review.ts
Agent-house (build profiling)	Scorea la traza de LLM-calls de cada build (Cost/Latency/Reliability/Context) + fixes rankeados	web/lib/swarm-profile.ts + run-trace.ts
SkillOpt	El skill DOC de cada rol es estado entrenable; un optimizer propone edits acotados (add/delete/replace), aceptados sólo si baten al actual en un validation set de 5 tasks congeladas; buffer de rechazos	web/lib/skill-evolution.ts , tablas puglit_skills / puglit_skill_rejects

El patrón común de toda la timeline: **medir por evidencia, no por mecanismo**. Cada mejora reemplaza una señal débil (compile verde, juez LLM solo, XP) por una fuerte (runtime smoke, validation set, constraint DB).

10.9 Weekly Insights

[Agregación futura — parcialmente automatizable hoy]. Los *weekly insights* se computan sobre puglit_metrics con la ventana que ya soporta metricRate(name, sinceDays) . El scorecard headline existe y agrega los cuatro KPIs que los críticos dijeron que hacen o rompen la tesis:

```
scorecard() → {
  build_success: { rate, n }, // first-build-success-rate
  smoke_pass: { rate, n }, // runtime smoke-pass-rate (0 5xx)
  judge_agreement: { rate, n }, // inter-judge agreement del panel
  ablation_swarm_win:{ rate, n }, // swarm vs single (ablación)
}
```

Un "weekly insight" se obtiene llamando metricRate(name, 7) para cada KPI y comparándolo contra la ventana de 30 días (default). La **diferencia semana-vs-mes** por KPI es el insight. Lo que **falta** para que sea automático es un job que persista el snapshot semanal (hoy scorecard() se calcula on-demand, no

se historiza en una tabla aparte). La materia prima (`puglit_metrics` , indexada por (`name`, `created_at DESC`)) ya está.

Insight semanal	Cómo se deriva	Estado
First-build-success ↑/↓	<code>metricRate("build_success", 7)</code> vs 30d	Calculable on-demand
Smoke-pass ↑/↓	<code>metricRate("smoke_pass", 7)</code>	Calculable on-demand
Acuerdo inter-juez	<code>metricRate("judge_agreement", 7)</code>	Calculable on-demand
Ablación swarm-win	<code>metricRate("ablation_swarm_win", 7)</code>	Calculable on-demand
Lecciones nuevas / semana	<code>COUNT(*) puglit_agent_diary WHERE created_at > now()-7d</code>	[Agregación futura]
Exemplars nuevos / semana	<code>COUNT(*) verified_exemplars ...</code>	[Agregación futura]

10.10 Monthly Insights

El default de `metricRate` ya es **30 días**, así que el scorecard mensual es de hecho el comportamiento por defecto de `scorecard()` . Los insights mensuales que la base de conocimiento puede sostener:

Insight mensual	Fuente	Estado
Scorecard de los 4 KPIs (rate + n)	<code>scorecard()</code> (default 30d)	Automático on-demand
Crecimiento del catálogo de módulos	<code>puglit_modules</code> (cuántos promovidos a <code>stable / core</code>)	[Agregación futura]
Evolución de skills aceptadas vs rechazadas	<code>puglit_skills.val_score</code> vs <code>puglit_skill_rejects</code>	[Agregación futura]
Subida de nivel agregada del roster	<code>puglit_agents.level/xp</code> (75 agentes)	Consultable, sin snapshot histórico
Brain growth	<code>brain-sync.ts</code> (UNION append-only / CRDT grow-set) + snapshots git/bucket	Persistido por snapshot, no por reporte

La persistencia mensual está garantizada a nivel de **datos crudos** por `brain-sync.ts` (tablas append-only unidas como grow-set CRDT; skills mutables arbitradas por `val_score` ; cloud Postgres autoritativo + snapshots a git/bucket vía `infra/brain-snapshot.sh`). Lo que aún no existe es la capa de **reporte** que destile esos datos en una narrativa mensual.

10.11 Trend Analysis

El análisis de tendencias se apoya en que **toda métrica es una serie temporal**: `puglit_metrics` guarda `(name, value, created_at)` y está indexada por `(name, created_at DESC)`, así que cualquier KPI es ploteable en el tiempo sin trabajo extra de esquema.

Tendencias que la arquitectura ya habilita:

1. **Curva de aprendizaje del enjambre** — `build_success` y `smoke_pass` en el tiempo: si la memoria funciona, deberían subir a medida que `verified_exemplars` y `puglit_agent_diary` crecen. Es la hipótesis central de Puglit, y es **medible** con la infra actual.
2. **Convergencia de jueces** — `judge_agreement` en el tiempo (el panel se vuelve más coherente).
3. **Ventaja del enjambre** — `ablation_swarm_win` sostenido > 0.5 valida swarm-vs-single.
4. **Maduración de skills** — `val_score` de `puglit_skills` por versión: cada versión aceptada batió a la anterior en el validation set de 5 tasks, así que la serie de versiones es monótona en val.

```
Serie temporal lista para tendencia (sin esquema nuevo):
puglit_metrics(name, value, created_at) → GROUP BY date_trunc('week', created_at)
                                         AVG(value) → tendencia por KPI
```

El plan de carga de la base. La tendencia más importante en curso es la **100-build brain-training batch** (`infra/train-brain.sh` + `brain-training-ideas.tsv`): apps diversas (juegos retro/Atari, calculadoras, trackers, verticales de negocio, money, catálogos, quizzes) que pre-cargan diary/exemplars/metrics/skills y se snapshotean a git. Esa batch es, literalmente, la generación del dataset sobre el cual el trend analysis pasa de "habilitado por la arquitectura" a "poblado con datos reales".

Resumen de honestidad. Medido y persistido hoy de forma automática: los 4 KPIs de `puglit_metrics` (vía `recordMetric` / `metricRate` / `scorecard`), los `verified_exemplars`, el `puglit_agent_diary` con anti-poisoning, los `puglit_modules` con lifecycle, y `puglit_skills` / `puglit_skill_rejects` con val-gating. Requiere agregación/reporte futuro: los rankings de uso por módulo, los conteos semanales/mensuales de lecciones y exemplars, los snapshots históricos del scorecard, y la narrativa de insights. La **materia prima** para todos ellos ya está en Postgres y indexada — falta el job que la destile, no los datos.